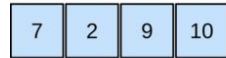


BUỔI 5 DATA SCIENCE 2

23/11/2025

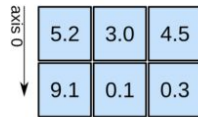
Numpy - pandas - Matplotlib

1D array



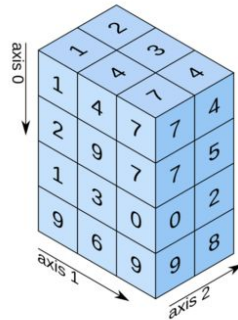
shape: (4,)

2D array



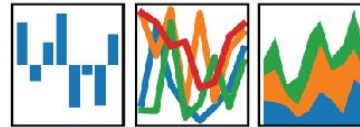
shape: (2, 3)

3D array



shape: (4, 3, 2)

pandas
 $y_i t = \beta' x_{it} + \mu_i + \epsilon_{it}$



	BandName	WavelengthMax	WavelengthMin
0	CoastalAerosol	450	430
1	Blue	510	450
2	Green	590	530
3	Red	670	640
4	NearInfrared	880	850
5	ShortWaveInfrared_1	1650	1570
6	ShortWaveInfrared_2	2290	2110
7	Cirrus	1380	1360

Numpy

❖ Định nghĩa:

- Thư viện xử lý số liệu nhanh (nhanh hơn list Python nhiều lần).
- Hỗ trợ toán học, thống kê, đại số tuyến tính.

❖ Đặc trưng

- Làm việc với ndarray (mảng nhiều chiều).
- Tất cả phần tử cùng kiểu dữ liệu.
- Tính toán theo kiểu vector hóa (không cần dùng vòng for).

```
import numpy as np
a = np.array([10, 20, 30])
b = np.array([1, 2, 3])
print(a + b)    # [11 22 33]
```

Numpy

Numpy nhanh hơn list??

- NumPy array lưu dữ liệu cùng kiểu và nằm liên tiếp trong bộ nhớ, còn list chứa nhiều kiểu khác nhau và nằm rải rác → truy cập chậm hơn.
- NumPy chia nhỏ công việc và xử lý song song, giúp tăng tốc đáng kể.
- NumPy sử dụng mã C, C++, Fortran tối ưu, chạy nhanh hơn rất nhiều so với Python thuần.

```
import time
n = 100_000
a = list(range(n))
b = list(range(n))
start = time.time()
c = [x + y for x, y in zip(a, b)]
end = time.time()
print(f"List time: {end - start:.4f} s")
```

List time: 0.0082 s

```
import numpy as np
import time
n = 100_000
a = np.arange(n, dtype=float)
b = np.arange(n, dtype=float)
start = time.time()
c = a + b          # 1 câu lệnh, không vòng for
end = time.time()
print(f"NumPy time: {end - start:.4f} s")
```

NumPy time: 0.0006 s

Numpy

❖ ndarray và Shape:

- Các cấp mảng

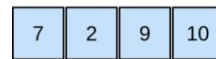
- + 0D: 2
- + 1D: [1,2,3]
- + 2D: [[1,2],[3,4]]
- + ND: nhiều chiều hơn.

- Thuộc tính quan trọng

- + `array.shape` → kích thước (hàng, cột).
- + `array.size` → tổng số phần tử.

- Ví dụ

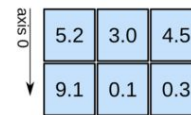
1D array



axis 0 →

shape: (4,)

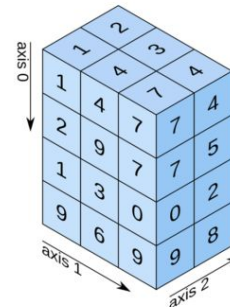
2D array



axis 1 →

shape: (2, 3)

3D array



shape: (4, 3, 2)

```
arr = np.array([[1, 2, 3], [4, 5, 6]])  
arr.shape      # (2, 3)  
arr.size       # 6
```

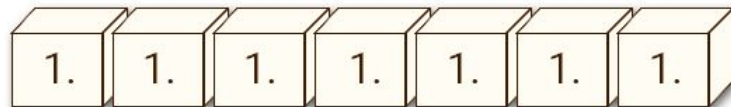
Numpy

❖ Tạo & thao tác mảng NumPy

- Các hàm tạo mảng

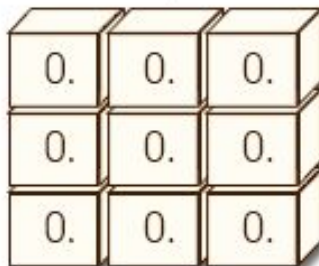
- + `np.array([...])`
- + `np.zeros((2,3))`
- + `np.ones((2,2))`
- + `np.full((2,2), 7)`
- + `np.empty((2,2))`

`np.ones(7)`



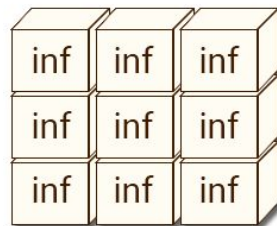
© w3resource.com

`np.zeros((3,3))`



© w3resource.com

`np.full((3,3), np.inf)`



© w3resource.com

Numpy

❖ Tạo & thao tác mảng NumPy

- Truy cập phần tử

	data
0	1
1	2
2	3

data[0]
1

data[1]
2

data[0:2]
1
2

data[1:]
2
3

data[-2:]
2
3

	data	
0	1	
1	2	-2
2	3	-1
3		

Numpy

❖ Tạo & thao tác mảng NumPy

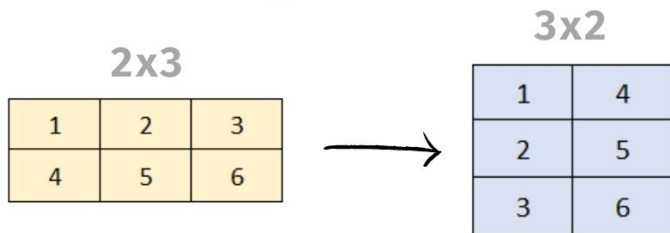
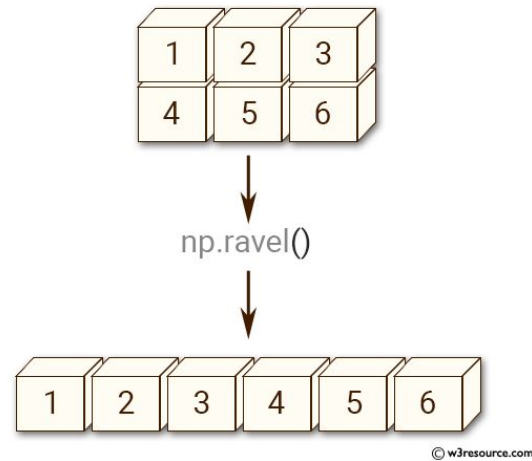
- Thay đổi cấu trúc

+ `arr.reshape((3,2))`

+ `arr.ravel()`

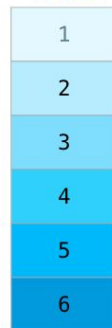
+ `arr.transpose()`

làm phẳng 1D

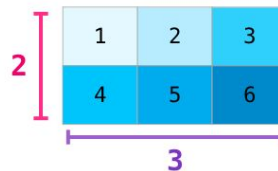


Transpose array

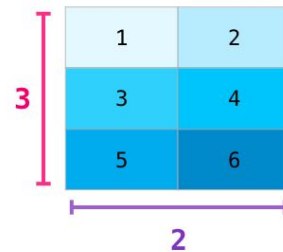
data



`data.reshape(2,3)`



`data.reshape(3,2)`



Numpy

❖ Các phép toán quan trọng

- Cơ bản

```
data = np.array([1, 2])
```

```
ones = np.ones(2, dtype=int)
```

+ data - ones

+ data + ones

+ data * data

+ data / data

data + **ones** =

data
1
2

 +

ones
1
1

 =

2
3

data - **ones** =

data
1
2

 -

ones
1
1

 =

0
1

data * **data** =

data
1
2

 *

data
1
2

 =

1
4

data / **data** =

data
1
2

 /

data
1
2

 =

1
1

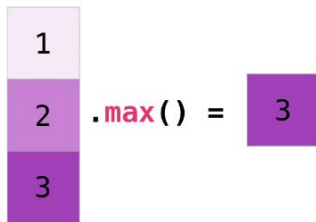
Numpy

❖ Các phép toán quan trọng

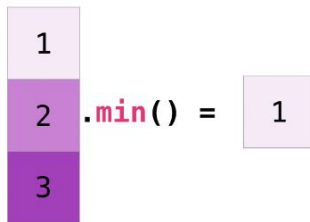
- Hàm toán học phổ biến

- + `np.sqrt(a)`
- + `np.sum(a)`
- + `np.max(a)`
- + `np.median(a)`
- + `np.dot(a,b)` # tích vô hướng

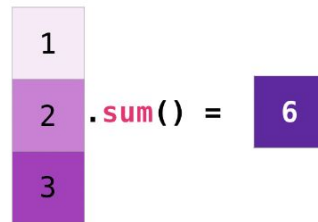
data



data



data



pandas

❖ Data Structures in Pandas

1. Pandas Series

- Cấu trúc dữ liệu **1 chiều**, có label (**index**).
- Lưu trữ được nhiều kiểu dữ liệu: int, float, string, object.
- Tạo **Series** từ: list, dict, NumPy array, hoặc file (CSV/Excel/SQL).
- Dùng để biểu diễn danh sách dữ liệu đơn lẻ.

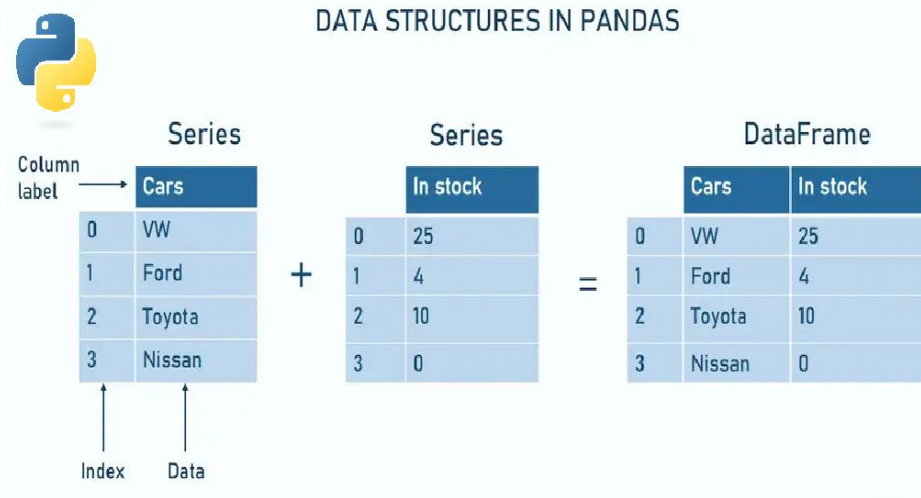
2. Pandas DataFrame

- Cấu trúc dữ liệu **2 chiều**, dạng **bảng với hàng và cột** (Excel)
- Tạo **DataFrame** từ: list, dict, list of dicts, hoặc file dữ liệu.
- Dùng để lưu trữ và thao tác dữ liệu dạng bảng trong phân tích dữ liệu.

pandas

❖ Data Structures in Pandas

Tiêu chí	Pandas Series	Pandas DataFrame
Số chiều	1 chiều	2 chiều
Hình dung	Giống 1 cột	Giống cả bảng Excel
Thành phần	Chỉ có index và data	Có hàng , cột , và index
Cấu tạo	Một dãy giá trị	Nhiều cột, mỗi cột là một Series
Mục đích	Lưu 1 danh sách dữ liệu	Lưu và phân tích dữ liệu dạng bảng



pandas

❖ Data Structures in Pandas

[6]: `import pandas as pd`

```
df = pd.DataFrame({  
    "Cars": ["VW", "Ford", "Toyota", "Nissan"],  
    "In stock": [25, 4, 10, 0]  
})
```

```
print(df)
```

	Cars	In stock
0	VW	25
1	Ford	4
2	Toyota	10
3	Nissan	0

```
i]: import pandas as pd  
  
# Series 1: Cars  
cars = pd.Series(["VW", "Ford", "Toyota", "Nissan"], name="Cars")  
  
# Series 2: In stock  
in_stock = pd.Series([25, 4, 10, 0], name="In stock")  
  
# Ghép 2 Series thành DataFrame  
df = pd.concat([cars, in_stock], axis=1)  
  
print("Series - Cars:")  
print(cars)  
print("\nSeries - In stock:")  
print(in_stock)  
print("\nDataFrame (Cars + In stock):")  
print(df)
```

```
Series - Cars:  
0      VW  
1     Ford  
2    Toyota  
3     Nissan  
Name: Cars, dtype: object
```

```
Series - In stock:  
0    25  
1     4  
2    10  
3     0  
Name: In stock, dtype: int64
```

```
DataFrame (Cars + In stock):  
      Cars  In stock  
0      VW        25  
1     Ford         4  
2    Toyota        10  
3     Nissan         0
```

pandas

❖ Indexing - Selecting - Slicing

1. Chọn dữ liệu bằng `[]`

Cú pháp	Ví dụ	Giải thích
<code>df["col"]</code>	<code>df["Age"]</code>	Chọn 1 cột . Dùng nhiều nhất.
<code>df[["col1", "col2"]]</code>	<code>df[["Age", "Salary"]]</code>	Chọn nhiều cột .

2. Chọn theo LABEL: `loc[]`

Cú pháp	Ví dụ	Giải thích
<code>df.loc[label]</code>	<code>df.loc["Avery Bradley"]</code>	Lấy 1 hàng theo tên.
<code>df.loc[[label1, label2]]</code>	<code>df.loc[["Avery Bradley", "R.J. Hunter"]]</code>	Lấy nhiều hàng theo tên.
<code>df.loc[[labels], [cols]]</code>	<code>df.loc[["Avery Bradley"], ["Team", "Salary"]]</code>	Lấy hàng + cột theo tên.
<code>df.loc[:, ["Team", "Salary"]]</code>	—	Tất cả hàng, chỉ 2 cột.

pandas

❖ Indexing - Selecting - Slicing

2. Chọn theo LABEL: `loc[]`

Cú pháp	Ví dụ	Giải thích
<code>df.loc[label]</code>	<code>df.loc["Avery Bradley"]</code>	Lấy 1 hàng theo tên.
<code>df.loc[[label1, label2]]</code>	<code>df.loc[["Avery Bradley", "R.J. Hunter"]]</code>	Lấy nhiều hàng theo tên.
<code>df.loc[[labels], [cols]]</code>	<code>df.loc[["Avery Bradley"], ["Team", "Salary"]]</code>	Lấy hàng + cột theo tên.
<code>df.loc[:, ["Team", "Salary"]]</code>	—	Tất cả hàng, chỉ 2 cột.

3. Chọn theo POSITION: `iloc[]`

Cú pháp	Ví dụ	Giải thích
<code>df.iloc[i]</code>	<code>df.iloc[3]</code>	Hàng thứ 3 (index-based).
<code>df.iloc[[i1, i2]]</code>	<code>df.iloc[[3,5,7]]</code>	Nhiều hàng theo vị trí.
<code>df.iloc[[rows], [cols]]</code>	<code>df.iloc[[1,3], [0,2]]</code>	Lấy theo số dòng + số cột.
<code>df.iloc[:, [0,2]]</code>	—	Chọn toàn bộ hàng nhưng chỉ 2 cột.

pandas

❖ Indexing - Selecting - Slicing

4. Slicing (Lấy đoạn)

(Giống *slicing list* Python)

Cú pháp	Ví dụ	Giải thích
<code>df.iloc[start:end]</code>	<code>df.iloc[0:4]</code>	Lấy hàng từ 0 → 3.
<code>df.iloc[:, 0:2]</code>	—	Lấy cột từ vị trí 0 → 1.

5. Lấy 1 giá trị (cell)

Cú pháp	Ví dụ	Giải thích
<code>df.iloc[i, j]</code>	<code>df.iloc[2,3]</code>	Lấy 1 ô theo vị trí.
<code>df.loc[row, col]</code>	<code>df.loc["V.Kohli", "Salary"]</code>	Lấy 1 ô theo tên hàng + cột.

pandas

❖ Indexing - Selecting - Slicing

6. Lọc dữ liệu bằng điều kiện (Boolean Filtering)

Các toán tử:

- So sánh: `<` `>` `<=` `>=` `==` `!=`
- Logic: `&` (AND); `|` (OR); `~` (NOT)

Cú pháp	Ví dụ	Giải thích
<code>df[df[col] > value]</code>	<code>df[df["Age"] > 30]</code>	Lọc theo điều kiện.
<code>df[(cond1) & (cond2)]</code>	<code>df[(df["Age"]>30) & (df["Salary"]>5e6)]</code>	Lọc nhiều điều kiện.

pandas

❖ Các thao tác DataFrame phổ biến

1. Xem nhanh dữ liệu

2. Thao tác cơ bản trên cột

Hàm	Giải thích
<code>df.head(n)</code> <code>df.tail(n)</code>	Xem n vài dòng đầu/ cuối để hiểu dữ liệu đang làm việc.
<code>df.info()</code>	Xem cấu trúc DataFrame: số cột, kiểu dữ liệu, giá trị thiếu.
<code>df.describe()</code>	Thống kê nhanh: min, max, mean, std (áp dụng cho dữ liệu số).

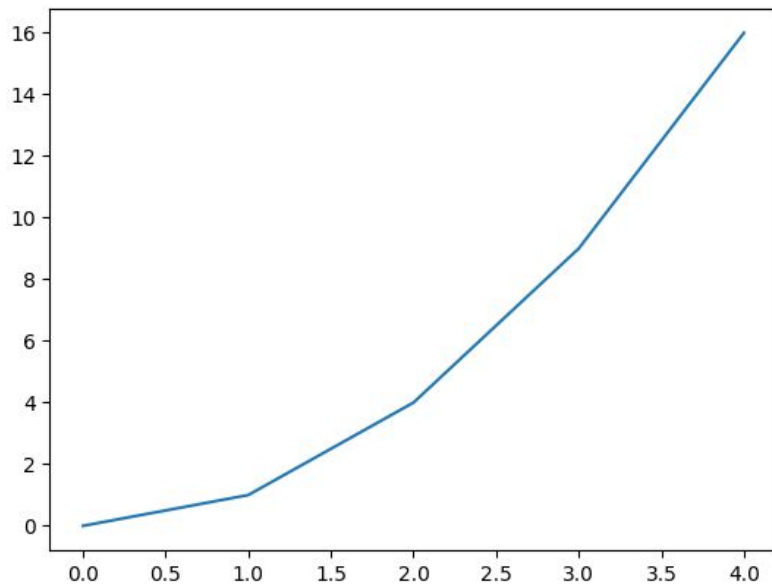
Thao tác	Cú pháp	Giải thích
Thêm cột	<code>df["new"] = ...</code>	Tạo một cột mới từ tính toán hoặc dữ liệu khác.
Xóa cột	<code>df.drop(columns=["col"])</code>	Xóa cột khỏi DataFrame.
Đổi tên cột	<code>df.rename(columns={"old": "new"})</code>	Đổi tên cột để dễ hiểu hoặc chuẩn hóa.
Sắp xếp dữ liệu	<code>df.sort_values("col")</code>	Sắp xếp DataFrame theo một cột.

Matplotlib

```
import matplotlib.pyplot as plt
```

```
x = [0, 1, 2, 3, 4]  
y = [0, 1, 4, 9, 16]
```

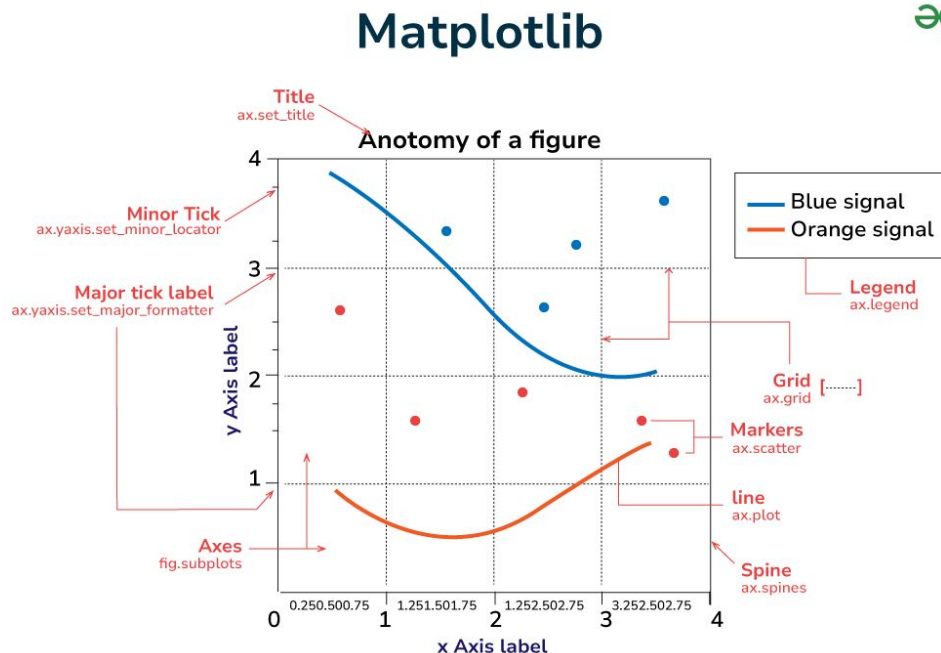
```
plt.plot(x, y)  
plt.show()
```



Matplotlib

❖ Các thành phần của Matplotlib

- **Figure** (khung hình): chứa toàn bộ các thành phần của biểu đồ.
- **Axes** (ô vẽ / subplot): khu vực hiển thị dữ liệu; một Figure có thể có nhiều Axes.
- **Axis** (trục x/y): quản lý giới hạn, vạch chia (ticks) và nhãn từng trục.
- **Lines & Markers** (đường & điểm): đường thể hiện xu hướng; điểm đánh dấu từng quan sát (ví dụ: scatter).
- **Title & Labels** (tiêu đề & nhãn):
- **Legend** (chú giải):
- **Lưu ý:** “Axes” ≠ “Axis”. “Axes” là ô vẽ; “Axis” là từng trục.



Matplotlib

❖ Quy trình dùng Matplotlib Pyplot

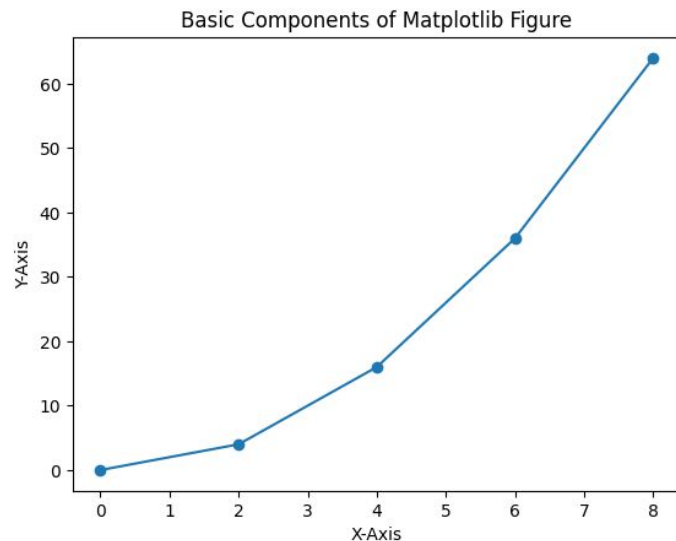
- **Import:** `import matplotlib.pyplot as plt`
- **Tạo dữ liệu:** list/array
- **Vẽ:** `plt.plot(...)` hoặc `ax.plot(...)`
- **Tùy biến:** `plt.title()`, `plt.xlabel()`,
`plt.ylabel()` (v.v.)
- **Hiển thị:** `plt.show()`

```
import matplotlib.pyplot as plt
```

```
x = [0, 2, 4, 6, 8]  
y = [0, 4, 16, 36, 64]
```

```
fig, ax = plt.subplots()  
ax.plot(x, y, marker='o', label="Data Points")  
  
ax.set_title("Basic Components of Matplotlib Figure")  
ax.set_xlabel("X-Axis")  
ax.set_ylabel("Y-Axis")
```

```
plt.show()
```



Matplotlib

❖ Các loại biểu đồ

- 1. Line Graph
- 2. Bar Chart
- 3. Histogram
- 4. Scatter Plot
- 5. Pie Chart
- ...

❖ Data Visualization in jupyter notebook

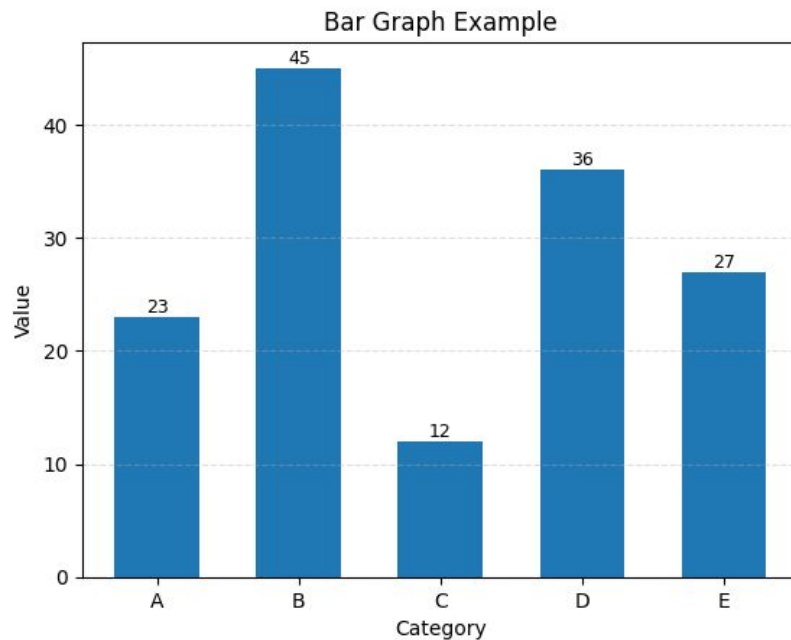
- `pip install matplotlib`
`pip install seaborn`
`pip install plotly`

Matplotlib

❖ Bar Graph

Các bước:

1. Import `matplotlib`
2. Chuẩn bị mảng **x** (nhãn) và **y** (giá trị)
3. Vẽ với `bar()`; có thể chỉnh `color`, `width`
4. Đặt **title/labels**
5. `plt.show()`



Matplotlib

❖ Pie Chart

Các bước:

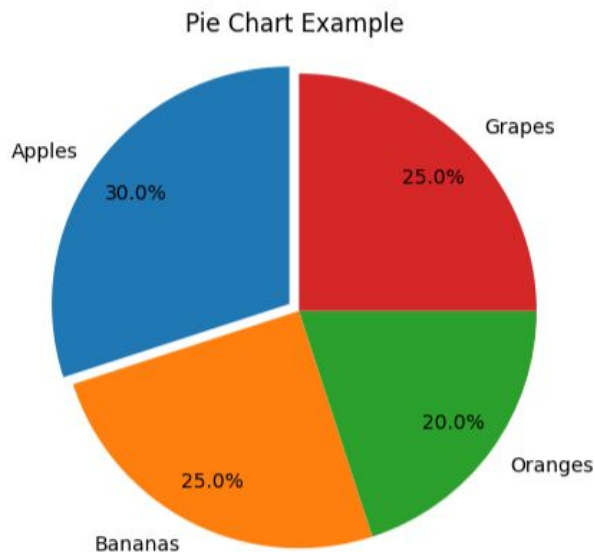
1. Import `matplotlib`
2. Chuẩn bị `labels` và `values`
3. Vẽ với `pie()`; có thể dùng `explode`, `autopct`, `pctdistance`, `startangle`
4. Đặt `title`
5. `plt.show()`

```
[4]: import matplotlib.pyplot as plt

labels = ['Apples', 'Bananas', 'Oranges', 'Grapes']
values = [30, 25, 20, 25]
explode = [0.05, 0, 0, 0] # làm nổi bật lát đầu tiên

fig, ax = plt.subplots()
ax.pie(values, labels=labels, explode=explode,
        autopct='%1.1f%%', startangle=90, pctdistance=0.8)
ax.set_title('Pie Chart Example')
ax.axis('equal') # giữ hình tròn

plt.show()
```



Matplotlib

❖ Pie Chart

- Seaborn: `pip install seaborn`
- Matplotlib thuần

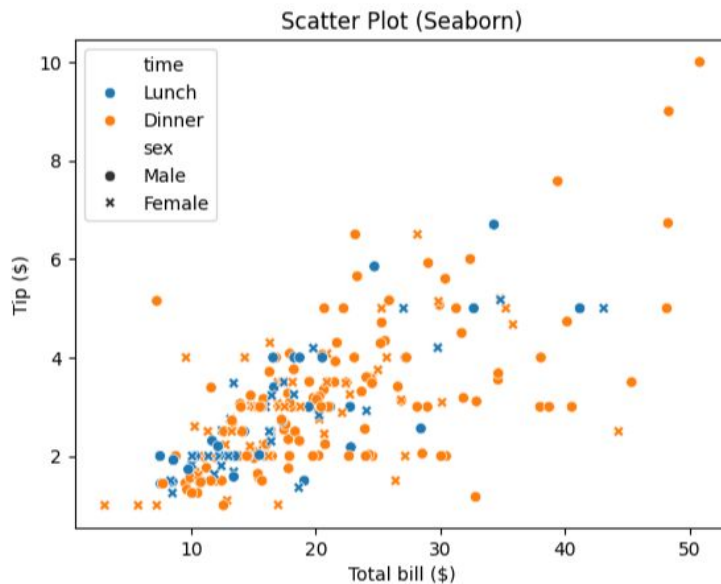
```
[11]: import seaborn as sns
import matplotlib.pyplot as plt

# Tải dữ liệu mẫu
tips = sns.load_dataset('tips')

fig, ax = plt.subplots()
sns.scatterplot(data=tips, x='total_bill', y='tip',
                hue='time', style='sex', ax=ax)

ax.set_title('Scatter Plot (Seaborn)')
ax.set_xlabel('Total bill ($)')
ax.set_ylabel('Tip ($)')

plt.show()
```



BÀI TẬP

[13.16 PRACTICE: Branches**: 24-hour time](#)

[13.18 PRACTICE: Branches**: Runway headings](#)

[13.35 PRACTICE: Loops**: Academic index score table](#)

[13.44 PRACTICE: Functions**: Rideshare pickup time](#)

[13.63 PRACTICE: Lists***: Grouping into ranges](#)

[13.61 PRACTICE: Lists**: Reverse in place](#)